

题目描述题解提交记录

LCR 010. 和为 K 的子数组

中等相关标签相关企业Aa

给定一个整数数组和一个整数 k ，请找到该数组中和为 k 的连续子数组的个数。

示例 1:

输入: nums = [1,1,1], k = 2
输出: 2
解释: 此题 [1,1] 与 [1,1] 为两种不同的情况

示例 2:

输入: nums = [1,2,3], k = 3
输出: 2

提示:

C++智能模式

```
1 class Solution {
2 public:
3     int subarraySum(vector<int>& nums, int k) {
4         unordered_map<int,int>map;
5         map[0] = 1;
6         int pre=0,count=0;
7         for(auto num:nums){
8             pre += num;
9             if(map.find(pre-k)!=map.end()){
10                 count += map[pre-k];
11             }
12             map[pre]++;
13         }
14         return count;
15     }
16 };
```

题目描述题解提交记录

LCR 011. 连续数组

中等相关标签相关企业Aa

给定一个二进制数组 $nums$ ，找到含有相同数量的 0 和 1 的最长连续子数组，并返回该子数组的长度。

示例 1:

输入: nums = [0,1]
输出: 2
解释: [0, 1] 是具有相同数量 0 和 1 的最长连续子数组。

示例 2:

输入: nums = [0,1,0]
输出: 2
解释: [0, 1] (或 [1, 0]) 是具有相同数量 0 和 1 的最长连续子数组。

提示:

- $1 \leq \text{nums.length} \leq 10^5$
- $\text{nums}[i]$ 不是 0 就是 1

C++智能模式

```
1 class Solution {
2 public:
3     int findMaxLength(vector<int>& nums) {
4         //前缀和加哈希表
5         int len = nums.size();
6         if(len == 0)return 0;
7         //申请哈希表
8         //key前缀和 value下标
9         unordered_map<int,int>mp;
10        mp[0] = -1;
11        //前缀和
12        int pre = 0;
13        //ans
14        int ans = 0;
15        for(int i = 0;i<len;i++){
16            //判断是否为0 为0的话加-1 将题目要求转换为和为0的最长连续子数组
17            pre += nums[i]>1:1;
18            //如果当前下标前缀和与之前下标前缀和相同 说明中间元素和为0 记录下中间元素数量
19            if(mp.find(pre) != mp.end()){
20                //哈希表的键是前缀和,但是下标
21                ans = max(ans,i-mp[pre]);
22            }else{
23                //只记录第一次出现的下标
24                mp[pre] = i;
25            }
26        }
27        return ans;
28    }
29 };
```

题库

题目描述题解提交记录

LCR 012. 寻找数组的中心下标

简单相关标签相关企业Aa

给你一个整数数组 $nums$ ，请计算数组的 **中心下标**。

数组 **中心下标** 是数组的一个下标，其左侧所有元素相加的和等于右侧所有元素相加的和。

如果中心下标位于数组最左端，那么左侧数之和视为 0 ，因为在下标的左侧不存在元素。这一点对于中心下标位于数组最右端同样适用。

如果数组有多个中心下标，应该返回 **最靠近左边** 的那一个。如果数组不存在中心下标，返回 -1 。

示例 1:

输入: nums = [1,7,3,6,5,6]
输出: 3
解释:
中心下标是 3。
左侧数之和 $\text{sum} = \text{nums}[0] + \text{nums}[1] + \text{nums}[2] = 1 + 7 + 3 = 11$ ，
右侧数之和 $\text{sum} = \text{nums}[4] + \text{nums}[5] = 5 + 6 = 11$ ，二者相等。

示例 2:

输入: nums = [1, 2, 3]
输出: -1
解释:
数组中不存在满足此条件的中心下标。

C++智能模式

```
3 int pivotIndex(vector<int>& nums) {
4     /*
5     计算“前缀和”与“后缀和”
6     然后用两个哈希表去存储两个“和”
7     哈希表 mp[0] = 0; 防止数组头尾是中心元素的可能
8     然后遍历其中一个哈希表看看是否有这样的元素
9     满足 i+1 == j-1 && mpPre[i] == mpBac[j]; return i+1;
10    如果没有这样的元素 return -1;
11    */
12    //优化为下面这种方法
13    //preSum[i] 下标i左侧元素的和
14    //bacSum[i] 下标i右侧元素的和
15    int len = nums.size();
16    if(len == 0)return -1;
17    vector<int> preSum(len+1, 0);
18    vector<int> bacSum(len+1, 0);
19    //下标为i左侧元素的和
20    for(int i = 1; i < len; ++i){
21        preSum[i] = preSum[i-1] + nums[i-1];
22    }
23    //下标为i右侧元素的和
24    for(int i = len-2; i >= 0; --i){
25        bacSum[i] = bacSum[i+1] + nums[i+1];
26    }
27    for(int i=0;i<len;++i){
28        if(preSum[i] == bacSum[i]){
29            return i;
30        }
31    }
32    return -1;
33 }
34 };
```

三道题大体上全都是采用前后缀的解题方式 个别配合哈希表解决

LCR010 求和为 k 的子数组 配合哈希表 map 键为前缀和 值为下标 解题原理就是

如果当前 i 处的前缀和 减去之前 j-1 处的前缀和是等于目标值 k 的 $pre[i] - pre[j-1] = k$ 那么这个 j 就是我要找的一个合法的数组 由于 $pre[i]$ 与 k 都是已知的 那么就是去找从 0 到 i 有几个这样的 j 计算公式为 $pre[j-1] = pre[i] - k$ 这时候就可以用哈希表记录当前前缀和 和此前缀和出现的次数 随后如果 find 到之后 $ans += map[j-1]$;

LCR011 求找到含有数量相同的 0 和 1 的长度最长的子数组

最初解题方案是将 0 当成 1 计算 计算出整体相加的和 最后 abs 一下 然后用 $\max(\text{zeroNum}, \text{oneNum}) - \text{abs}(\text{sum})$; 但是有用例不通过 这个公式未经推导, 无法使用 是用来求不连续数组的

之后采用了改进方法,同样是前缀和配合哈希表 如果当前位置 i 与前某个位置 j 前缀和相同 那么就证明 j-1~i 之间和为 0 正满足题目要求, 用一个 ans 动态更新最大长度

LCR012 寻找中间下标, 采用前后缀和的形式, 前缀求的是 i 的左侧的和,后缀求的是 i 的右侧的和 如果 $pre[i] == \text{bac}[i]$ 那么当前值为中点

优化: 首先计算出整体和 Sum 与 $pre[]$ $\text{Sum} - pre[i] - \text{nums}[i] == pre[i]$ 就说明此处为值中间下标